MCMC Remediation Design Doc

Branch: `aris/bayesian-review` **Worktree:** `.worktrees/aris-bayesian-review`
Date: 2026-04-20 **Status:** APPROVED — ready for overnight launch prompt

---

Goal

Determine whether the MCMC sampler and the enumeration engine produce compatible posteriors when run on the SAME hypothesis space (depth-6, no injections, uniform grammar). This is a self-contained methodological validation step that does NOT touch the existing depth-7 injected model.

What this does NOT do

- Does NOT overwrite the existing depth-7 enumeration + injections model
- Does NOT claim depth-6 agreement implies depth-7 agreement
- Does NOT attempt to make MCMC discover injected hypotheses
- Does NOT modify the production model used for adversarial hand generation

---

Design Decisions

1. Hypothesis space

Parameter	Value	Rationale
Depth	6	Matches MCMC hard cap (max_depth=6). Fair comparison.
Max programs (enum)	300,000	Good class coverage at depth-6; runs parallel with MCMC so no wall-clock penalty.
Injections	None	MCMC cannot discover injections by random walk. Fair comparison requires same reachable space.
Grammar	Uniform	Both methods use the same flat grammar prior.

2. MCMC configuration

Parameter	Value	Rationale
n_steps	50,000	5x current (10k). Enough for convergence diagnostics.
n_chains	4	Independent chains, no communication. Same as current.
max_depth	6	Match enumeration depth.
max_nodes	25	Same as current.
noise_epsilon	0.01	Same as current.
top_k	UNLIMITED	Serialize ALL visit_counts. The Night 2 truncation bug is the #1 problem — eliminate it.
beta_start	0.3	Annealing: start warm for exploration.
beta_end	1.0	Anneal to true posterior by end. Acceptance uses annealed score; visit_counts track unannealed.
seed	42	Reproducible. Per-chain seeds: 42, 43, 44, 45.

3. Convergence checkpoints

- **12 checkpoints** evenly spaced over 50k steps (at steps 4167, 8333, 12500, ..., 50000)
- At each checkpoint, record:
  - Cumulative visit_counts snapshot
  - Top-50 hypotheses by visit frequency
  - Total unique programs visited so far
  - Acceptance rate in preceding interval
- Post-hoc diagnostics (computed from checkpoints):
  - KL divergence between consecutive checkpoints (should decrease)
  - Spearman rho of top-50 ranking stability
  - Top-K overlap (Jaccard) between consecutive checkpoints
  - mass_in_full_list trajectory (should increase as more programs accumulate visits)

4. Extension estimation

Use case	Method	Probes
Shared (Question A)	Adaptive ladder	100k base; escalate to 1M for classes with base_rate < 0.001
Native MCMC (Question B)	MCMC's built-in	10k probes (as currently implemented)

Both are recorded. Question A isolates inference differences. Question B tests full pipeline.

5. Comparison framework

Two-layer reporting per rule:

Layer 1 — Coverage: - `mass_in_full_list`: fraction of MCMC total visits that map to an enumeration equivalence class - `n_unmatched_programs`: MCMC programs that fail parse/fingerprint/class-lookup - `unmatched_mass_fraction`: visit weight in unmatched programs

Layer 2 — Agreement on shared support: - `mean_|delta|`: mean absolute difference in p_accept across 5000 uniform probe hands - `max_|delta|`: worst-case disagreement - `spearman_rho`: rank correlation of probe acceptance probabilities - `top_K_overlap`: Jaccard overlap of top-50 most-accepted probes - `KL_divergence`: KL(enum || mcmc) on shared-support acceptance distribution

Validity gating (pre-registered thresholds from Night 2):

```
VALIDITY_THRESHOLDS = {
    "min_mass_in_full_list":        0.90,
    "min_mass_in_top_k":            0.80,
    "max_mass_dropped_parse":       0.05,
    "min_enum_retained_mass":       0.95,
    "min_probe_hit_rate_either":    0.001,
    "max_predicate_exceptions":     0,
}
```

6. Class aggregation

MCMC produces per-program-string visit counts. Enumeration produces per-equivalence-class posteriors. To compare:

1. Parse each MCMC program string into AST
2. Compute extensional fingerprint (same method as enumeration)
3. Look up fingerprint in enumeration's class table
4. Sum visit weights per class
5. Normalize to get MCMC class-level posterior

Programs that fail any step (parse error, unknown fingerprint) go into `unmatched` bucket.

7. Rule selection

20 rules from the 61-rule gallery catalogue. Selection criteria: - Include all 7 from Night 2 comparison (all_red, all_same_suit, all_even, triple_any_adjacent, strict_increasing, four_of_a_kind_adjacent, ranks_palindrome) - Add 13 more spanning: easy/high-base-rate, medium, hard/rare, structural/adjacency patterns - Cover range of extension sizes (from very common rules to very rare ones)

Exact list to be finalized in launch prompt (will pick from gallery_rules.py).

8. Output structure

```
review-stage/experiments/night2/mcmc_remediation/
  config.json                    # Full run config (reproducibility)
  enum_depth6_300k/
    pool.json                    # Equivalence classes + fingerprints
    extensions.json              # Per-class extension estimates (adaptive ladder)
    posteriors/                  # Per-rule posterior vectors
  mcmc_50k_4chains/
    raw_visits/                  # Per-rule full visit_counts (unlimited)
    checkpoints/                 # 12 snapshots per rule
    native_extensions/           # MCMC's 10k-probe estimates
  comparison/
    question_a/                  # Shared extensions comparison
    question_b/                  # Native extensions comparison
    convergence_diagnostics/     # Checkpoint-derived metrics
    summary.json                 # Per-rule validity flags + metrics
```

9. Framing for paper

If comparison_valid = True on most rules: > "At depth-6 with uniform grammar, MCMC and enumeration produce compatible posteriors (mean|delta| < X, rho > Y), validating methodological consistency of the two inference engines."

If comparison_valid = False: > "The validity-gated comparison identified [specific failure mode]. We document this as an open limitation and specify concrete next steps."

Either outcome is publishable. The framework catches its own failures.

Estimated runtime

Component	Estimate
Enumeration (300k, depth-6, 20 rules)	~30-60 min
Extension estimation (adaptive ladder)	~20-40 min
MCMC (50k steps x 4 chains x 20 rules)	~8-12 hours
Comparison + diagnostics	~10 min
Total wall clock	**~9-13 hours**

Enumeration runs first (needed for class lookup table), then MCMC runs with class aggregation happening at each checkpoint. Or: enumeration and MCMC run in parallel, with aggregation as a post-processing step after both complete.

Key fixes from Night 2

1. **Truncation bug eliminated**: full visit_counts serialized (no top_k cap)
2. **Class aggregation**: MCMC strings mapped to enum classes before comparison
3. **Depth parity**: both at depth-6
4. **Injection fairness**: neither method has injections
5. **Extension parity**: shared high-precision estimates for Question A
6. **Convergence tracking**: 12 checkpoints with diagnostic metrics
7. **Annealing**: beta_start=0.3 for better exploration

Dependencies

- `src/gallery_analysis/mcmc_search.py` — needs modification to:
  - Accept beta_start/beta_end for annealing schedule
  - Serialize full visit_counts (remove top_k truncation on output)
  - Emit checkpoint snapshots at configurable intervals
- `src/gallery_analysis/analyze_mcmc.py` — fix hardcoded top_k=20 at line 136
- `review-stage/experiments/night2/compare_enum_vs_mcmc.py` — extend with:
  - Class aggregation step
  - Two-layer reporting
  - Convergence diagnostics from checkpoints
  - Question A vs B separation
- New: enumeration runner at depth-6/300k (no injections, no adaptive ladder on enum side — just flat grammar)

Risks

1. **50k steps may still not converge** for hard rules. Mitigated by: convergence checkpoints will show whether we're still improving at step 50k, informing whether a longer run is needed.
2. **Class aggregation parse failures** could be high if MCMC visits many malformed programs. Mitigated by: unmatched bucket tracking; if >5% mass is unmatched, the validity flag trips.
3. **Memory**: 50k steps x 4 chains x 20 rules with full visit_counts could be large. Mitigated by: serialize per-rule (not all in memory at once), use gzip on output.